

ALGORITHM REPORT

Missing Integer

1- Iterative (Non-Recursive)

Pseudocode

Function missingInteger_iterative(A[0 .. N-1])

size = N + 2

exist[0 .. size-1] = all 0

For i = 0 to N-1:

 If A[i] > 0 And A[i] < size:

 exist[A[i]] = 1

 End If

End For

For i = 1 to size-1:

 If exist[i] = 0:

 Return i

 End If

End For

Analysis

Basic Operation: The comparison if (A[i] > 0 && A[i] < size)

Best Case

Loop 1: N + 2

Loop 2: N

Loop 3: 1

$$\text{Total} = 2N + 3 \Rightarrow O(N)$$

The Best Case occurs when only number 1 is not in the array.

Worst Case

Loop 1: $N + 2$

Loop 2: N

Loop 3: $N + 1$

$$\text{Total} = 3N + 3 \Rightarrow O(N)$$

The Worst Case occurs when the array contains exactly 1, 2, 3, ..., N.

2- Recursive

Pseudocode

Algorithm main()

```
{
    write "enter array size:"
    read n
    for i = 0 to n-1 do
    {
        write "enter element at index", i, ":"
        read A[i]
    }
    result := missingInteger_recursive(A, n)
    write "the smallest missing positive integer is:", result
}
```

Algorithm missingInteger_recursive(A, n)

```
{
    return findMissing(A, n, 1)
}
```

Algorithm findMissing(A, n, current)

```
{
    if current > n + 1 then
        return current
    if isFound(A, n, current) = 0 then
        return current
    return findMissing(A, n, current + 1)
}
```

Algorithm isFound(A, n, val)

```
{  
    if n = 0 then return 0  
    if A[0] = val then  
        return 1  
    return isFound(A+1, n-1, val)  
}
```

Analysis

Basic Operation: The comparison if (A[0] = val) inside isFound

isFound — Recurrence Relation

In function isFound, each recursive call decrements n by 1. The recurrence is:

$$\begin{aligned}T(n) &= T(n-1) + c \\&= T(n-2) + 2c \\&= T(n-3) + 3c \\&= T(1) + (n-1)c \\&= O(n)\end{aligned}$$

findMissing — Recurrence Relation

In findMissing, for every value of current, isFound is called ($O(n)$ each). The recurrence is:

$$\begin{aligned}T(n) &= T(n-1) + n \\&= T(n-2) + (n-1) + n \\&= T(n-3) + (n-2) + (n-1) + n \\&= n(n+1)/2 = O(n^2)\end{aligned}$$

Time Complexity of overall algorithm: $O(n^2)$

Best & Worst Cases

Best Case: If 1 is the missing positive integer $\rightarrow O(n)$

Worst Case: If the array contains all numbers from 1 to n $\rightarrow O(n^2)$

3- Comparison

4-	Non-Recursive	Recursive
Basic Operation	Comparison (in Loop 2)	Comparison (in "isFound")
Best Case	$O(n)$	$O(n)$
Worst Case	$O(n)$	$O(n^2)$
Speed	Faster	Slightly Slower
Memory Usage	Low	High

Notes

Non-Recursive: Both best and worst cases run in $O(n)$ — the iterative approach is consistently linear.

Recursive: Best case is $O(n)$ when 1 is the missing integer. Worst case degrades to $O(n^2)$ when the array contains all numbers 1 to n , because `isFound` is called $O(n)$ times, each taking $O(n)$.